PROYECTOSISTEMAS DE SUBASTAS CONCURRENTES

Módulo: Programación de Servicios y Procesos (2025-2026)

Autores:
Adrian Laga Lambea y Rubén Pasamón Finol
01/03/2026

índice

1. Propuesta de trabajo **3**
Descripción General 3
Objetivos del Proyecto 3
Alcance 3
Requisitos Funcionales y No Funcionales 3
2. Planificación **4**
Arquitecturas y Tecnologías Usadas 4
Diseño Inicial 4
Fases del Proyecto 6
3. Roles y Organización de la Pareja **7**
4. Evaluación del Proceso **7**
5. Evaluación de Resultado **8**

1. Propuesta de trabajo

Descripción General

El problema que resuelve la aplicación es el siguiente: permite la compra y venta de artículos mediante un sistema de subastas, asegurando que las pujas concurrentes se procesen de forma justa, segura y sin inconsistencias de datos. Está pensada para usuarios particulares que desean vender artículos de segunda mano o comprar productos al mejor precio mediante pujas. No obstante, en la aplicación los usuarios podrán registrarse, publicar artículos indicando un precio base y un tiempo de expiración, y pujar en tiempo real por los artículos de otros usuarios.

Objetivos del Proyecto

El proyecto pretende desarrollar una aplicación totalmente funcional basada en una arquitectura Cliente-Servidor robusta en Java. A través de ella podremos gestionar el acceso concurrente a recursos compartidos como el estado y precio de la subasta, mediante mecanismos de sincronización. Además para garantizar la seguridad de las comunicaciones hemos utilizado sockets seguros con TLS.

Alcance

La aplicación incluye el registro/login de usuarios, creación de subastas con temporizador, listado de subastas activas, sistema de pujas en tiempo real con notificaciones a los clientes conectados, y cierre automático de la subasta declarando un ganador.

Por otro lado, no incluye la pasarela de pago real, un sistema de envío/logística, panel de administración web complejo, ni recuperación de contraseñas por email.

Requisitos Funcionales y No Funcionales

- Funcionales:
    - RF01: El sistema debe permitir el registro de usuarios validando nombre de usuario, email y contraseña.
    - RF02: El sistema debe permitir iniciar sesión con usuario y contraseña, generando un token de sesión único.
    - RF03: El sistema debe permitir cerrar sesión, invalidando el token.
    - RF04: El sistema debe permitir publicar un artículo con precio inicial, descripción y duración.
    - RF05: El sistema debe listar todas las subastas activas ordenadas por tiempo restante.
    - RF06: El sistema debe mostrar el detalle de una subast.
    - RF07: El servidor debe rechazar pujas menores o iguales a la puja actual y validar que el pujador no sea el vendedor.
    - RF08: El sistema debe notificar en tiempo real a todos los clientes cuando hay una nueva puja válida.
    - RF09: El servidor debe cerrar la subasta al expirar el tiempo y notificar al vendedor y al ganador.

- No Funcionales:
    - RNF01: Seguridad: Toda la comunicación debe ir cifrada sobre TLS (SSLSocket / SSLServerSocket).
    - RNF02: Seguridad: Las contraseñas deben almacenarse con hash SHA-256 y salt, nunca en texto plano.
    - RNF03: Seguridad: Se deben usar PreparedStatement para prevenir inyecciones SQL.
    - RNF04: Concurrencia: Sincronización estricta para evitar condiciones de carrera si dos usuarios pujan al mismo tiempo.
    - RNF05: Rendimiento: El servidor debe soportar al menos 50 conexiones simultáneas.
    - RNF06: Rendimiento: Baja latencia en la actualización de pujas y notificaciones.
    - RNF07: Persistencia: Los datos deben almacenarse en una base de datos SQLite.
    - RNF08: Rendimiento: El monitoreo de subastas expiradas debe ejecutarse cada segundo.
    - RNF09: Compatibilidad: El sistema debe funcionar en JDK 17 o superior.
    - RNF10: Interoperabilidad: El protocolo de comunicación debe ser JSON sobre TCP.

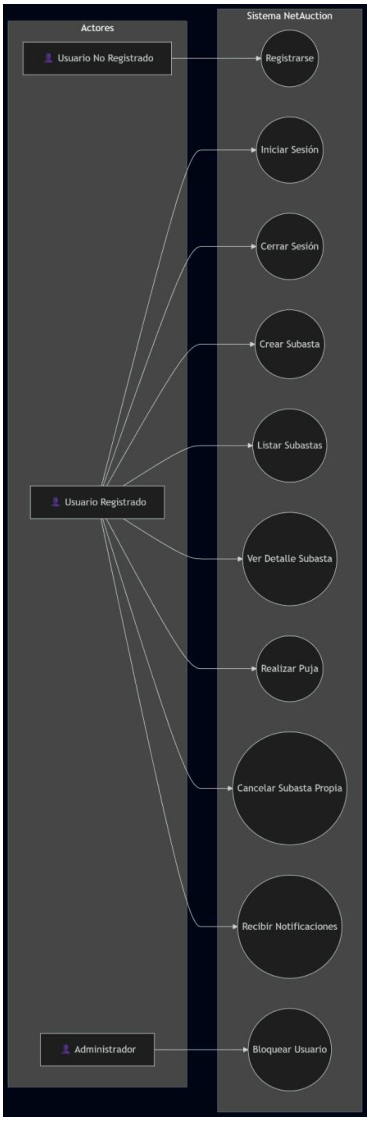
2. Planificación

Arquitecturas y Tecnologías Usadas

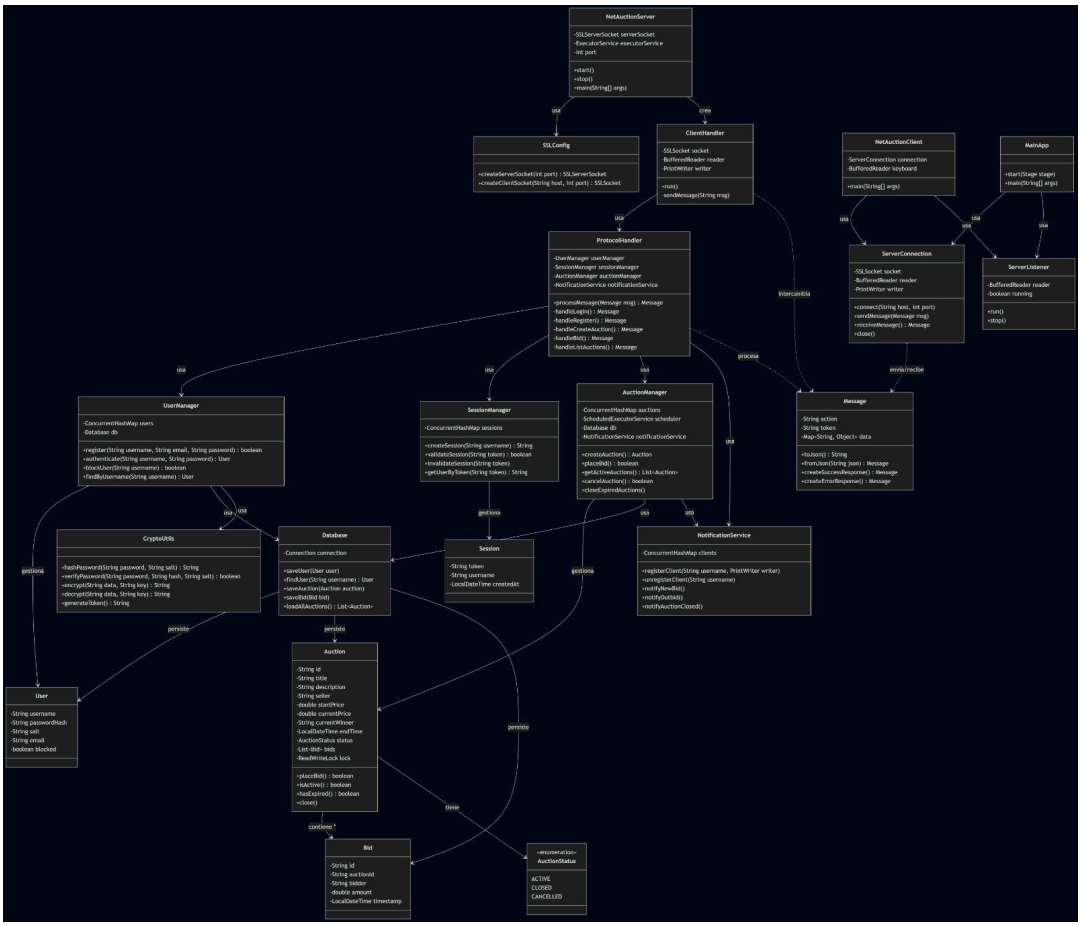
La aplicación utiliza el lenguaje Java SE con una arquitectura Cliente-Servidor multihilo. Las comunicaciones se realizan a través de sockets TCP cifrados. Otro concepto muy importante a destacar es la concurrencia, utilizamos la clase Thread Pools para manejar múltiples clientes, y ScheduledExecutorService para los temporizadores de las subastas. Y por último a mencionar, el almacenamiento lo haremos a través de una base de datos relacional SQLite para persistir usuarios y subastas, accedida mediante JDBC.

Diseño Inicial

- **Diagrama de Casos de Uso:**



- **Diagrama de Clases:**



Fases del Proyecto

1. Análisis y Diseño (Dia 1-2): Los primeros días los dedicamos a definir los protocolos de mensajes JSON entre cliente y servidor, la estructura de clases y la organización del proyecto en dos módulos independientes (server-app y client-app). También diseñamos el modelo de datos (usuarios, subastas, pujas) y decidimos usar SQLite como base de datos por su sencillez y no requerir un servidor externo.
2. Infraestructura de Red (Dia 3-4): Montamos el servidor básico con sockets TCP, la conexión de clientes mediante hilos y la configuración completa de los certificados TLS. Generamos el keystore del servidor con keytool, exportamos el certificado público y creamos el truststore para el cliente. Una vez configurado, toda la comunicación entre cliente y servidor quedó cifrada con TLSv1.2/1.3.
3. Lógica Concurrente (Dia 5-6): Implementamos la clase Auction con ReadWriteLock para garantizar que las pujas simultáneas se procesen de forma atómica y sin condiciones de carrera. También desarrollamos los temporizadores con ScheduledExecutorService que revisan cada segundo si alguna subasta ha expirado, cerrándola automáticamente y notificando al vendedor y al ganador.
4. Interfaz y Notificaciones (Dia 7-8): Desarrollamos el cliente de consola y el cliente GUI con JavaFX. Además implementamos el sistema de notificaciones push mediante un hilo ServerListener que escucha mensajes asíncronos del servidor, permitiendo que los clientes conectados reciban en tiempo real las alertas de nuevas pujas (NEW_BID), pujas superadas (OUTBID) y cierre de subastas (AUCTION_CLOSED).
5. Pruebas y Documentación (Dia 9-10): Realizamos pruebas con múltiples clientes conectados simultáneamente para verificar que las pujas concurrentes se procesan correctamente sin inconsistencias. También probamos el cierre automático de subastas y las notificaciones. Por último, redactamos la memoria final del proyecto.

3. Roles y Organización de la Pareja

Desde el principio decidimos repartirnos el trabajo de forma clara para no pisarnos. Adrían se encargó principalmente de toda la parte del servidor: la conexión con los clientes, la lógica de las subastas, la base de datos y la configuración de los certificados TLS. Ruben se centró más en la parte del cliente: tanto la versión de consola como la interfaz gráfica con JavaFX, y en que las notificaciones llegasen bien en tiempo real.

Dicho esto, hay partes que hicimos entre los dos, como por ejemplo el diseño del protocolo de mensajes JSON, los modelos de datos (User, Auction, Bid, Message) y las pruebas finales de integración, donde necesitábamos estar los dos para lanzar el servidor y conectar varios clientes a la vez y comprobar que todo funcionaba correctamente.

Nos organizamos principalmente en clase, aprovechando las horas de prácticas para avanzar juntos y resolver dudas sobre la marcha. Fuera de clase nos comunicamos por WhatsApp para coordinarnos si uno avanzaba por su cuenta en casa. Utilizamos Git para gestionar el código y así evitar conflictos a la hora de juntar lo que cada uno iba haciendo.

No seguimos ninguna metodología formal, simplemente íbamos hablando qué tocaba hacer cada semana y cada uno se ponía con su parte. Cuando uno terminaba algo, el otro lo revisaba por encima y si había algún problema lo comentábamos y lo arreglábamos entre los dos.

4. Evaluación del Proceso

Evaluación del proceso de desarrollo

En general el desarrollo siguió la planificación que habíamos marcado, diríamos que en un 80%. La parte donde más nos atascamos fue en la fase de sincronización, ya que las pruebas con pujas concurrentes resultaron ser más complejas de lo que esperábamos y tuvimos que dedicarle más tiempo del previsto.

Problemas detectados y soluciones

El primer problema que nos encontramos fue con las pujas concurrentes. Cuando dos usuarios pujaban prácticamente al mismo tiempo, el servidor dejaba pasar ambas pujas con la misma cantidad porque se producía una condición de carrera. Lo solucionamos utilizando ReadWriteLock en el objeto Auction, de forma que las lecturas podrían ser concurrentes pero las escrituras de pujas fueran exclusivas.

El segundo problema fue con los certificados TLS. Al principio no teníamos claro cómo funcionaba keytool ni la diferencia entre el keystore del servidor y el truststore del cliente, y nos daba errores al conectar. Tuvimos que dedicar una sesión extra a investigarlo y nos retrasó un día, pero al final conseguimos que toda la comunicación quedase cifrada correctamente.

5. Evaluación de Resultado

Evaluación de la funcionalidad

El sistema funciona como esperábamos. El temporizador de las subastas se ejecuta correctamente en segundo plano usando un ScheduledExecutorService que comprueba cada segundo si alguna subasta ha expirado. Cuando el tiempo llega a cero, la subasta se cierra automáticamente, se declara un ganador y se rechazan todas las pujas que lleguen después. Esto lo probamos creando subastas cortas de un minuto y comprobando que efectivamente se cerraban en el momento justo.

El registro y login de usuarios funciona bien, las contraseñas se almacenan con hash SHA-256 y salt por lo que nunca se guardan en texto plano en la base de datos. La creación de subastas, el listado de las activas y el sistema de pujas funcionan correctamente tanto desde el cliente de consola como desde la interfaz gráfica.

Cumplimiento de requisitos

En cuanto a si se cifran las comunicaciones, sí, toda la comunicación entre cliente y servidor va cifrada mediante TLSv1.2/1.3 usando SSLSocket y SSLServerSocket. El cliente valida el certificado del servidor a través de su truststore antes de enviar ningún dato.

Respecto a la sincronización, sí, las pujas concurrentes se procesan de forma atómica gracias al ReadWriteLock. Lo comprobamos conectando varios clientes a la vez y lanzando pujas simultáneas para verificar que no se producían condiciones de carrera ni datos inconsistentes.

Y sobre las notificaciones en tiempo real, también funcionan correctamente. Cuando un usuario puja, todos los clientes conectados a esa subasta reciben una notificación con el nuevo precio. Si a un usuario le superan la puja recibe un “outbid”, y cuando la subasta se cierra tanto el vendedor como el ganador recibe un “auction_closed”. Todo esto llega en tiempo real gracias al hilo ServerListener que escucha constantemente los mensajes del servidor.